Day 146 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 7 reference

Watch: How Stable Diffusion Works: AI Image Generation Explained (CodeLucky) | Coding Stable Diffusion from scratch in PyTorch (Umar Jamil)

`day146_text_to_digit_capstone.py`, segment by segment, in the order `main()` uses things. The week's parts are imported, not pasted; the clash check, the zero-shot variant, the prompt normaliser and the front door are new. The networks (Day 126), scorer (Day 138), nearest-digit distance (Day 141), DDIM and grey share (Day 142), encoders (Day 143) and the text-conditioned model with guidance (Day 144) are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: w = guidance scale · a, b = guesses with and without the prompt · N = samples per prompt

Segment 0 — Setup

Imports: the week's parts, as modules

CODE

```
sys.path.insert(0, HERE)
for w in ("week-20-generative-models-gans", "week-19-unsupervised-learning-ii",
    "week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day139_gan_build_and_forward_diffusion as d139
    import day141_exercise as d141
    import day142_exercise as d142
    import day143_exercise as d143
    import day144_exercise as d144
```

- `with contextlib.redirect_stdout(io.StringIO()):` — each exercise runs its own checks when imported; their printing goes into a string that is thrown away.
- `import day143_exercise as d143` — as a module, Day 143's functions keep Day 143's constants (its ACTS); pasted into this file they would not.

Constants

CODE

```
HOLD = (7, 1)
N = 200
W_GUIDE = 2.0
STEP_LIST = (10, 25, 50, 100)
NEAR = 2.0
```

- `STEP_LIST = (10, 25, 50, 100)` — a tuple of the DDIM step counts tried in section 4.
- `W_GUIDE = 2.0` — Day 144's best guidance scale, used for every request.

Segment 1 — The new parts

clashes: reading constants with ast

CODE

```
def clashes(days=range(140, 146)):
    seen = {}
    for d in days:
        src = open(os.path.join(HERE, f"day{d}_exercise.py"), encoding="utf-8").read(); tree =
            ast.parse(src)
        last = max(n.lineno for n in tree.body if isinstance(n, ast.FunctionDef))
        for n in tree.body:
            if isinstance(n, ast.Assign) and n.lineno < last and isinstance(n.targets[0],
                ast.Name) and n.targets[0].id.isupper():
                seen.setdefault(n.targets[0].id, []).append((d, ast.get_source_segment(src,
                    n.value)))
    return {k: v for k, v in seen.items() if len({s for _, s in v}) > 1}
```

- `ast.parse(src)` — the file's text as a tree of statements, without running it.
- `max(n.lineno for n in tree.body if isinstance(n, ast.FunctionDef))` — the line of the last function: constants after it belong to the file's checks, not its parts.
- `n.targets[0].id.isupper()` — only names written in capitals: the convention for constants.
- `ast.get_source_segment(src, n.value)` — the value exactly as written in the file, e.g. '["relu", "linear"]'.
- `len({s for _, s in v}) > 1` — a set of the written values: more than one means the files disagree.

zero_shot_acts: the same encoders, another activation list

CODE

```
def zero_shot_acts(img, txt, X, y, acts):
    T = d126.forward(txt, d143.bow([f"the digit {w}".split() for w in d143.NUM]),
        acts)[-1].astype(np.float64)
    Z = d126.forward(img, X, acts)[-1].astype(np.float64)
    zero = float(np.mean(np.vstack([T, Z]) == 0))
    T /= np.linalg.norm(T, axis=1, keepdims=True) + 1e-12; Z /= np.linalg.norm(Z, axis=1,
        keepdims=True) + 1e-12
    return float(((Z @ T.T).argmax(1) == y).mean()), zero
```

- `d126.forward(txt, ..., acts)[-1]` — the last layer's output; forward pairs layers with acts by zip, so a longer list is cut short without an error.
- `np.mean(np.vstack([T, Z]) == 0)` — the share of entries that are exactly 0: a ReLU on the last layer sets every negative entry to 0.
- `+ 1e-12` — keeps a row of all zeros from being divided by zero.

normalise and generate: the front door

MATH

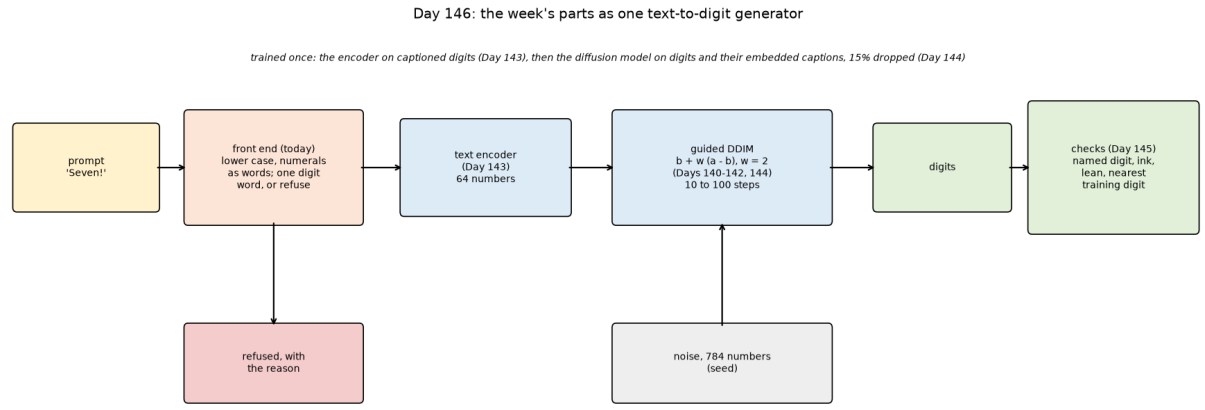
$$\text{guided } x_0 = b + w\,(a - b) \quad \text{(Day 144)}$$

CODE

```
def normalise(prompt):
    words = re.findall(r"[a-z0-9]+", prompt.lower())
    words = [d143.NUM[int(w)] if len(w) == 1 and w.isdigit() else w for w in words]
    return [w for w in words if w in d143.IDX], [w for w in words if w not in d143.IDX]
def generate(txt, ema, prompt, n=N, w=W_GUIDE, negative=None, steps=100, seed=1):
    known, unknown = normalise(prompt)
    digits = sorted({k for k in known if k in d143.NUM}, key=d143.NUM.index)
    if len(digits) != 1:
        return None, "refused: " + ("no digit word it knows" if not digits else "two digits (" + "
            and ".join(digits) + ")")
    c = d143.embed_texts(txt, [known])[0]
    cn = None if negative is None else d143.embed_texts(txt, [normalise(negative)[0]])[0]
    note = "drawn" + (f"; ignored {', '.join(repr(u) for u in unknown)}" if unknown else "")
    return d144.cfg_sample(ema, c, n, w, cn, steps, seed), note
```

- `re.findall(r"[a-z0-9]+", prompt.lower())` — runs of letters and digits; punctuation and spaces separate words.
- `d143.NUM[int(w)] if len(w) == 1 and w.isdigit() else w` — a single numeral becomes its word; anything else is kept.
- `sorted({...}, key=d143.NUM.index)` — a set removes repeats; sorting by position in NUM gives a fixed order.
- `if len(digits) != 1: return None, "refused: ..."` — two return values every time: images or None, and a note saying what happened.
- `d144.cfg_sample(ema, c, n, w, cn, steps, seed)` — Day 144's guided DDIM sampler, unchanged.

SYSTEM DIAGRAM



The week's parts as one generator.

Segment 2 — The experiments

main: shared pieces, trained once

CODE

```
X, y = d126m.mnist(); X2 = (X * 2 - 1).astype(np.float32)
ink, slant, thick, lean = d143.measure(X, y)
te = 60000 + np.random.RandomState(0).permutation(10000)[:2000]
t0 = time.perf_counter(); body, head = d139.train_classifier(X[:60000], y[:60000])
sc = d139.Scorer(body, head, X[:10000], X[10000:30000])
t1 = time.perf_counter(); img, txt, _ = d143.train_clip(X[:60000], y[:60000], thick[:60000],
    lean[:60000], hold=HOLD)
print(f"(feature network for the scores: {t1 - t0:.0f} s; text and image encoders:
    {time.perf_counter() - t1:.0f} s)\n")
```

- `te = 60000 + np.random.RandomState(0).permutation(10000)[:2000]` — the same 2,000 test digits Day 143 used, so its number can be checked.
- `hold=HOLD` — the same call as on Day 144, so the same encoders come out.

demo_model: reading earlier days' outputs

CODE

```
z = d143.zero_shot(img, txt, X[te], y[te])
old = float(re.search(r"zero-shot accuracy of the held-out encoders (\d\.\d+)",
    open(os.path.join(HERE, "day143_run_output.txt"), encoding="utf-8").read()).group(1))
print(f"text encoder (Day 143's train_clip, the held-out pair): zero-shot accuracy {z:.4f}; Day
    143's run printed {old:.4f}: "
      f"{'the same' if f'{z:.4f}' == f'{old:.4f}' else 'DIFFERENT'}")
t0 = time.perf_counter(); net, ema, log = d144.train_cond(X2[:60000], y[:60000], thick[:60000],
    lean[:60000], txt)
print(f"text-conditioned diffusion model (Day 144's train_cond, {log[-1][0]:,} steps): loss
    {log[0][1]:.4f} -> {log[-1][1]:.4f}, "
      f"{time.perf_counter() - t0:.0f} s")
yy = np.repeat(np.arange(10), N)
t0 = time.perf_counter(); F = np.vstack([d144.cfg_sample(ema, d143.embed_texts(txt, [[w]])[0], N,
    W_GUIDE, seed=1 + k) for k, w in enumerate(d143.NUM)])
took = time.perf_counter() - t0; i, fid, p, r, _ = sc(F); acc = float(np.mean(d144.said(sc, F) ==
    yy))
row = f"{W_GUIDE:5.1f} {acc:12.3f} {i:6.2f} {fid:8.2f} {p:10.3f} {r:7.3f}
    {d142.grey_share(F):11.4f} {F.mean():6.3f}"
old = next(ln for ln in open(os.path.join(HERE, "day144_run_output.txt"),
    encoding="utf-8").read().split("\n") if ln.startswith("  2.0 "))
print(f"prompts 'zero' ... 'nine', {N} samples each, w = 2, DDIM 100 steps, the same starting
    noise as Day 144:")
print(f"{'':>9}{'w':>5} {'right digit':>12} {'IS':>6} {'FID':>8} {'precision':>10} {'recall':>7}
    {'grey share':>11} {'ink':>6}")
print(f"{'today':>9}{row}   ({took:.0f} s)")
print(f"{'Day 144':>9}{old[:len(row)]}")
print(f"every printed digit the same as Day 144's run: {row == old[:len(row)]}")
```

- `re.search(r"... (\d\.\d+)", text).group(1)` — the number captured by the brackets, as text; float() makes it a number.
- `next(ln for ln in ... if ln.startswith(" 2.0 "))` — the first line of Day 144's output that starts like the w = 2 row.
- `old[:len(row)]` — Day 144's row without its time column, which is not expected to match.
- `row == old[:len(row)]` — two strings compared character by character: True only if every printed digit agrees.

demo_front: raw against the front end

CODE

```
for prompt, d in tests:
    F = d144.cfg_sample(ema, d143.embed_texts(txt, [prompt.split()])[0], N, W_GUIDE); lab =
        d144.said(sc, F)
    cnt = np.bincount(lab, minlength=10); top = np.argsort(-cnt)[:2]
    G, note = generate(txt, ema, prompt)
    rr = "-" if d is None else f"{np.mean(lab == d):.3f}"
    fr = "-" if G is None or d is None else f"{np.mean(d144.said(sc, G) == d):.3f}"
    print(f"{prompt!r:>24} {rr:>11} {d143.NUM[top[0]] + ' ' + str(cnt[top[0]]) + ', ' +
        d143.NUM[top[1]] + ' ' + str(cnt[top[1]]):>22}   {note:<40} {fr:>6}")
    keep[prompt] = (F[:6], None if G is None else G[:6], note)
```

- `[prompt.split()]` — the raw route, as on Day 144: the prompt split on spaces and embedded as typed.
- `G, note = generate(txt, ema, prompt)` — the front-end route: two values unpacked; G is None when the request is refused.
- `{prompt!r:>24}` — !r prints the string with its quotes; >24 right-aligns it in 24 characters.

demo_steps: the same requests with fewer steps

CODE

```
for s in STEP_LIST:
    if s == 100:
        F, took = F100, t100
    else:
        t0 = time.perf_counter(); F = np.vstack([d144.cfg_sample(ema, d143.embed_texts(txt,
            [[w]])[0], N, W_GUIDE, steps=s, seed=1 + k) for k, w in enumerate(d143.NUM)])
        took = time.perf_counter() - t0
    _, fid, _, r, _ = sc(F); acc = float(np.mean(d144.said(sc, F) == yy)); rows.append((s, acc,
        fid, took)); keep[s] = F[::N // 4][:40]
    print(f"{s:6d} {acc:12.3f} {fid:8.2f} {r:7.3f} {d142.grey_share(F):11.4f} {took:6.1f}s {1000 *
        took / len(F):13.2f}")
```

- `if s == 100: F, took = F100, t100` — 100 steps are section 2's settings, so its samples and time are reused.
- `steps=s, seed=1 + k` — fewer steps, the same starting noise for each digit.

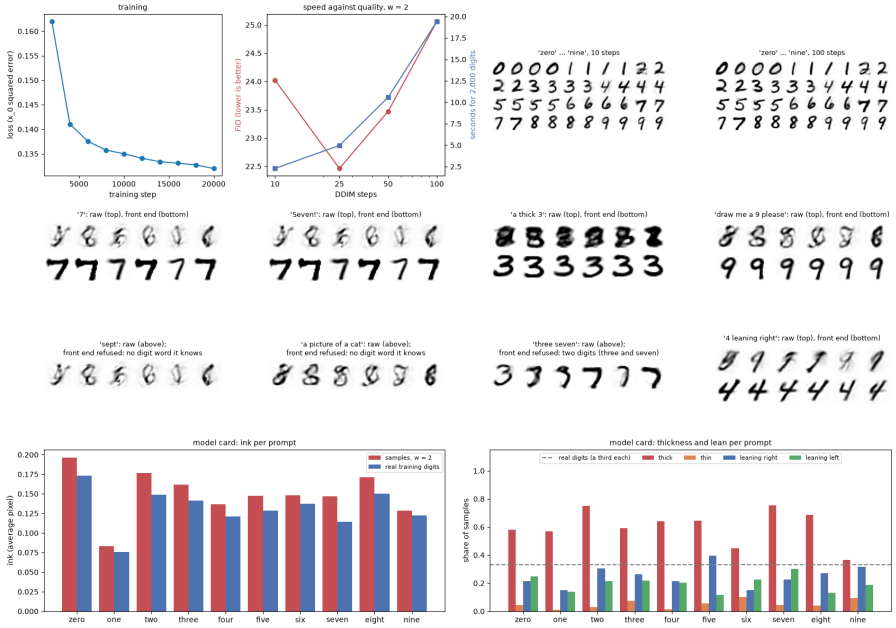
demo_card: one row per digit

CODE

```
for d in range(10):
    k = slice(d * N, (d + 1) * N); tr = y[:60000] == d
    a1, a2 = np.percentile(ink[:60000][tr], [100 / 3, 200 / 3]); b1, b2 =
        np.percentile(slant[:60000][tr], [100 / 3, 200 / 3])
    c = (np.mean(lab[k] == d), ik[k].mean(), ink[:60000][tr].mean(), np.mean(ik[k] > a2),
        np.mean(ik[k] < a1), np.mean(sl[k] > b2),
         np.mean(sl[k] < b1), np.median(dF[k]), np.median(dT[y[te] == d]), np.mean(dF[k] < NEAR));
            card.append(c)
```

- `k = slice(d * N, (d + 1) * N)` — a slice object: the 200 rows that belong to this digit's prompt.
- `np.percentile(ink[:60000][tr], [100 / 3, 200 / 3])` — Day 143's cut points: the thirds of this digit's training images.
- `np.median(dT[y[te] == d])` — the same distance for real test digits of this kind: what an honest new digit looks like.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `ast.parse(text)` — a file's statements as a tree, without running them.
- `ast.get_source_segment(text, node)` — the exact text of one part of the tree.
- `re.findall(pattern, s)` — every match, as a list of strings.
- `re.search(pattern, s).group(1)` — the text captured by the first brackets.
- `next(x for x in it if cond)` — the first item that passes a test.
- `{x!r}` — an f-string field shown with repr(): strings keep their quotes.
- `slice(a, b)` — a reusable a:b index.
- `return None, note` — a function that returns a result and a reason together.

End of Day 146 syntax reference. For the experiments and what they showed, see the Day 146 complete guide.